16.8 Λυμένες Ασκήσεις

16.8.1 Οι εξαιρέσεις ως μέρος της κανονικής ροής

Να αναπτύξετε συνάρτηση που μετράει πόσες φορές υπάρχει κάθε γράμμα του αγγλικού αλφάβητου, ανεξαρτήτως αν είναι κεφαλαίο ή πεζό, σε ένα αρχείο κειμένου. Να κληθεί η συνάρτηση στην main και να εμφανιστεί το αποτέλεσμα.

Λύση

Ο μηχανισμός των εξαιρέσεων μπορεί να χρησιμοποιηθεί και σε άλλες περιπτώσεις πέραν του ελέγχου των λαθών. Η λύση που προτείνουμε σε αυτήν την εφαρμογή αξιοποιεί τον μηχανισμό των εξαιρέσεων για να εξαιρέσει τους χαρακτήρες του κειμένου που δεν είναι γράμματα του αγγλικού αλφάβητου.

```
import java.io.File;
import java.util.Scanner;

public class CountLetters {

    public static String loadFile(String pathName) {
        StringBuilder sB = new StringBuilder();
        try ( Scanner input = new Scanner(new File(pathName))) {
            while (input.hasNextLine()) {
                sB.append(input.nextLine() + "\n");
            }
        } catch (Exception e) {
            System.out.println("File " + pathName + " not found");
            System.exit(1);
        }
        return new String(sB);
    }

    public static int[] countLetters(String data) {
        int[] counter = new int[26];
        data = data.toUpperCase();

        for (int i = 0; i < data.length(); i++) {
            try {
                counter[data.charAt(i) - 'A']++;
            } catch (Exception e) {

            }
        }
        return counter;
    }

    public static void main(String[] args) {
        String data = loadFile("data" + File.separator + "text01.txt");
        int[] counter = countLetters(data);
        for (int i = 0; i < counter.length; i++) {
            System.out.println((char) ('A' + i) + " " + counter[i]);
        }
    }
}
```

Κώδικας 16.12 *Εξαίρεση των χαρακτήρων που δεν είναι γράμματα του αγγλικού αλφάβητου*

Καταρχάς, η loadFile φορτώνει σε ένα String, τα δεδομένα του αρχείου που βρίσκεται στο pathName. Σε περίπτωση που αποτύχει, βγάζει κατάλληλο μήνυμα και τερματίζει η εφαρμογή.

Στη συνέχεια, η countLetters χρησιμοποιεί έναν πίνακα ακέραιων, μήκους 26, δηλαδή διαθέτει μία θέση για κάθε γράμμα του αγγλικού αλφάβητου. Στη συνέχεια, μετατρέπει όλους τους χαρακτήρες του κειμένου σε

κεφαλαίους. Χαρακτήρες που δεν διακρίνονται σε κεφαλαία ή πεζά, π.χ. η τελεία, δεν επηρεάζονται από αυτήν τη μετατροπή. Μετά, μέσα στην επαναληπτική διαδικασία προσπελαύνει ένα προς ένα τους χαρακτήρες και επιχειρεί να αυξήσει τον μετρητή στην αντίστοιχη θέση του αρχείου. Αν ωστόσο ο χαρακτήρας δεν αντιστοιχεί σε γράμμα του Αγγλικού αλφάβητου, τότε η έκφραση data.charAt(i) - 'A' βγαίνει έξω από τα όρια του πίνακα counter. Στην περίπτωση αυτή παράγεται ArrayOutOfIndexException. Ο διαχειριστής εξαιρέσεων που ακολουθεί συλλαμβάνει οποιαδήποτε εξαίρεση παραχθεί μέσα στο try. Όμως δεν κάνει κάποια περαιτέρω διαχείριση, απλώς με τη σύλληψή της η εξαίρεση καταστέλλεται. Έτσι η επαναληπτική διαδικασία συνεχίζεται και μετράει μόνο τους χαρακτήρες που μας ενδιαφέρουν.

Τέλος, στην main εκτυπώνονται οι χαρακτήρες μαζί με τον αντίστοιχο μετρητή.

16.8.2 Εξίσωση β' βαθμού

Να υλοποιηθεί η κλάση QuadraticEquation που αναπαριστά εξισώσεις δευτέρου βαθμού. Να υποστηρίζει μεθόδους υπολογισμού της διακρίνουσας και των ριζών x1 και x2. Να ελεγχθεί με κατάλληλες εξαιρέσεις. Στην main να αναπτυχθεί στοιχειώδης κώδικας ελέγχου.

Λύση

```
public class QuadraticEquation {

    private final double a, b, c;

    public QuadraticEquation(double a, double b, double c) {
        if (Lib.approximateEquals(a, 0)) {
            throw new IllegalArgumentException();
        }
        this.a = a;
        this.b = b;
        this.c = c;
    }

    public QuadraticEquation(double a, double b) {
        this(a, b, 0);
    }

    public QuadraticEquation(double a) {
        this(a, 0, 0);
    }

    public double discriminant() {
        return Math.pow(b, 2) - 4 * a * c;
    }

    @Override
    public String toString() {
        return "QuadraticEquation{" + "a=" + a + ", b=" + b + ", c=" + c +
'}';
    }


    public double x1() {
        double disc = discriminant();
        if (disc < 0) {
            throw new RuntimeException(this+", discriminant < 0");
        }
        return (-b + Math.sqrt(disc)) / (2 * a);
    }

    public double x2() {
```

```
            double disc = discriminant();
            if (disc < 0) {
                throw new RuntimeException(this+", discriminant < 0");
            }
            return (-b - Math.sqrt(disc)) / (2 * a);
        }

        public static void main(String[] args) {
//          QuadraticEquation qE1=new QuadraticEquation(2,4,6),
//                  qE2=new QuadraticEquation(1,4),
//                  qE3=new QuadraticEquation(2);
//          System.out.println(qE1.discriminant());
//          System.out.println(qE2.discriminant());
//          System.out.println(qE3.discriminant());
//          System.out.println(qE1.x1()+" "+qE1.x2());
//          System.out.println(qE2.x1()+" "+qE2.x2());
//          System.out.println(qE3.x1()+" "+qE3.x2());

            QuadraticEquation qE=new QuadraticEquation(1, 1, -1);
            System.out.println(qE.discriminant());
            System.out.println(qE.x1()+" "+qE.x2());
        }
    }
```

Κώδικας 16.13 *Η κλάση QuadraticEquation*

Καταρχάς, έχουμε τρεις δημιουργούς στην κλάση ώστε εύκολα ο χρήστης της να μπορεί να δημιουργήσει εξισώσεις που όλοι οι συντελεστές είναι διάφοροι του μηδενός, εξισώσεις όπου ο συντελεστής c είναι 0 και εξισώσεις όπου b και c είναι μηδέν. Εξισώσεις στις οποίες ο b είναι μηδέν και ο c διάφορος του μηδενός δημιουργούνται αναγκαστικά με τον πρώτο δημιουργό, καθώς οι παράμετροι στην Java είναι παράμετροι θέσης (positional parameter) και δεν μπορεί να υποστηριχτεί διαφορετικός δημιουργός.

Στην εξίσωση δευτέρου βαθμού, ισχύει εξ ορισμού $\alpha \neq 0$. Έτσι, αν επιχειρηθεί η δημιουργία αντικειμένου QuadraticEquation με $\alpha == 0$, ο δημιουργός παράγει μη ελεγχόμενη εξαίρεση τύπου IllegalArgumentException. Κάτι τέτοιο είναι πιθανό να συμβεί από κάποιο λάθος στον υπολογισμό του συντελεστή a σε άλλο σημείο της εφαρμογής. Ωστόσο, αν το λάθος φτάσει μέχρι τον δημιουργό της QuadraticEquation, δεν μπορούμε να κάνουμε πλέον κάτι και ως εκ τούτου επιλέξαμε την καταλληλότερη προαιρετικά ελεγχόμενη εξαίρεση την οποία δεν ελέγχουμε και προκαλεί τερματισμό της main.

Στον υπολογισμό των ριζών επίσης έχουμε τη μη ελεγχόμενη εξαίρεση RuntimeException. Σε μια τέτοια περίπτωση, θα πρέπει η τεκμηρίωση της κλάσης να αναφέρει ρητά πως οι x1 και x2 καλούνται μόνο μετά από έλεγχο πως η discriminant επιστρέφει αριθμό μεγαλύτερο ή ίσο με το 0. Παρόλα αυτά, ο χρήστης της κλάσης ενδέχεται να καλέσει την x1 ή την x2 χωρίς να ελέγξει. Για αυτήν την περίπτωση πρέπει να υπάρχει έλεγχος μέσα στις x1 και x2. Εδώ επίσης διακόπτουμε τη ροή της εφαρμογής. Μια εναλλακτική λύση θα ήταν να επιστρέφουμε Double.NaN.

Ο σχολιασμένος κώδικας μέσα στην main επιδεικνύει την παραγωγή εξαίρεσης (εφόσον αποσχολιαστεί) στην περίπτωση που ζητηθούν οι ρίζες, ενώ η διακρίνουσα είναι μικρότερη του μηδενός.

16.9 Ασκήσεις προς λύση

16.9.1 Άμεση πρόσβαση με διαχείριση εξαιρέσεων

Στον κώδικα 15.6, ανάγνωσης και εγγραφής σε αρχεία με τυχαία προσπέλαση, προσθέστε κατάλληλη διαχείριση των πιθανών εξαιρέσεων. Αντικαταστήστε τις εξαιρέσεις που ήδη περιλαμβάνονται στον κώδικα με καταλληλότερες.

16.9.2 Περιοδικός Πίνακας με διαχείριση εξαιρέσεων

Στην κλάση TableOfChemichalElements που υλοποιεί τη διεπαφή Cloneable προσθέστε διαχείριση εξαιρέσεων.

16.9.3 grep με διαχείριση εξαιρέσεων

Στη συνάρτηση public static void grep(File file, String searchItem) της άσκησης 15.8.2 προσθέστε κατάλληλη διαχείριση εξαιρέσεων.

16.9.4 Quiz με εξαιρέσεις οριζόμενες από τον χρήστη

Τροποποιήστε το quiz της άσκησης 15.8.1 ώστε να υποστηρίζει εξαιρέσεις. Ελέγξτε τα πιθανά λάθη στη δομή των ερωτήσεων με κατάλληλες εξαιρέσεις οριζόμενες από τον χρήστη.

Βιβλιογραφία

[1] C. Horstmann, Η γλώσσα προγραμματισμού Java, Αναλυτική Προσέγγιση. Broken Hill Publishers, 2021.

[2] J. Farrell, Java, Εκμάθηση με πρακτικά παραδείγματα. Κριτική, 2018.

[3] "The try-with-resources Statement (The Java™ Tutorials > Essential Java Classes > Exceptions)." https://docs.oracle.com/javase/tutorial/essential/exceptions/tryResourceClose.html (accessed Jan. 12, 2022).

Κεφάλαιο 17

Σύνοψη
Στην ενότητα αυτή παρουσιάζονται οι γενικεύσεις της Java. Πιο συγκεκριμένα παρουσιάζονται οι γενικευμένες κλάσεις, οι γενικευμένες συναρτήσεις, τα όρια των γενικεύσεων, η κληρονομικότητα μεταξύ γενικευμένων τύπων και η αξιοποίηση του χαρακτήρα μπαλαντέρ.

Προαπαιτούμενη γνώση
Η ενσωμάτωση, η κληρονομικότητα και ο πολυμορφισμός στο αντικειμενοστρεφές μοντέλο με Java, οι αφηρημένες κλάσεις καθώς και οι βασικές προκαθορισμένες κλάσεις, δηλαδή οι String, StringBuilder, ArrayList, LinkedList, Stack, PriorityQueue, HashSet, HashMap, HashTree, LocalDate, Localtime, Period και Duration. Οι διεπαφές στην Java. Η βασική διαχείριση των αρχείων. Ο μηχανισμός των εξαιρέσεων.

Λέξεις κλειδιά
Γενίκευση (generic), παραμετροποιημένος τύπος (parametarized type), μπαλαντέρ (wildcard), πρωταρχικός τύπος (raw type).

17 Γενικεύσεις

Ήδη έχουμε χρησιμοποιήσει εκτεταμένα τις γενικεύσεις στην απλή μορφή τους. Οι γενικεύσεις αποτελούν ένα ισχυρό προγραμματιστικό εργαλείο που δίνει τη δυνατότητα ανάπτυξης γενικευμένων αλγορίθμων, δηλαδή αλγορίθμων που επενεργούν σε δεδομένα διαφορετικών τύπων, ενώ παράλληλα διατηρούν τον αυστηρό χαρακτήρα στον έλεγχο των τύπων. Επιπλέον διευκολύνουν την ανάπτυξη καθώς αφαιρούν την ανάγκη για μετατροπή τύπου και καθιστούν τον κώδικα ευανάγνωστο.

```
import java.util.ArrayList;

public class Benefits {

    public static void rawType() {
        ArrayList l = new ArrayList();
        l.add(1);
        l.add("Jim");
        l.add(new Object());
        int i = (Integer) l.get(0);
        String s = (String) l.get(1);
        Object o = l.get(2);
        System.out.println(i);
        System.out.println(s);
        System.out.println(o);
    }

    public static void generic() {
        ArrayList<String> l = new ArrayList<>();
        l.add("Jim");
        String s = l.get(0);
        System.out.println(s);
    }

    public static void main(String[] args) {
        rawType();
    }
}
```

Κώδικας 17.1 *Πρωταρχικός και γενικευμένος τύπος της ArrayList*

Στον κώδικα 17.1 χρησιμοποιούμε την κλάση ArrayList ως πρωταρχικό (raw type) και γενικευμένο τύπο. Στη συνάρτηση rawType χρησιμοποιείται ως πρωταρχικός τύπος. Σε αυτή τη μορφή της, ο τύπος των αντικειμένων που αποθηκεύει είναι Object. Καθώς η Object είναι γονική κλάση όλων των τύπων στην Java, το πραγματικό αντικείμενο μπορεί να είναι οποιασδήποτε κλάσης. Έτσι στην rawType αποθηκεύουμε έναν Integer, ένα String και ένα Object. Επομένως κατά την αποθήκευση των δεδομένων δεν μπορεί να γίνει έλεγχος του τύπου, καθώς όλες οι κλάσεις είναι αποδεκτές. Επιπλέον κατά την ανάκτηση πρέπει να γνωρίζουμε τι τύπο έχουμε αποθηκεύσει σε κάθε θέση του ArrayList ώστε να προβούμε στην κατάλληλη μετατροπή τύπου.

Αντίθετα, στη συνάρτηση generic δηλώνουμε ένα ArrayList που δέχεται αποκλειστικά String. Επομένως, υπάρχει η δυνατότητα πλήρους ελέγχου του τύπου κατά την αποθήκευση των δεδομένων, ενώ παράλληλα κατά την ανάκτησή τους είναι γνωστός ο τύπος τους, οπότε η μετατροπή τύπου είναι περιττή.

Ο όρος πρωταρχικός τύπος αναφέρεται σε γενικευμένο τύπο που χρησιμοποιείται χωρίς παράμετρο τύπου. Δεν ισχύει όμως για τύπους που δεν αποτελούν γενικεύσεις. Οι γενικεύσεις προστέθηκαν στην Java στην έκδοση 5. Πριν από αυτήν, ο μόνος τρόπος για να χρησιμοποιηθεί ένα ArrayList ήταν αυτός που επιδεικνύεται στη συνάρτηση rawType. Μετά την προσθήκη των γενικεύσεων, οι πρωταρχικοί τύποι διατηρήθηκαν μόνο για λόγους συμβατότητας με τους κώδικες που είχαν ήδη αναπτυχθεί, ωστόσο δεν συνίσταται η χρήση τους σε νέους κώδικες.

17.1 Ορισμός γενικευμένων κλάσεων

Μία γενικευμένη κλάση είναι στην ουσία μια κλάση παραμετροποιημένη προς έναν ή περισσότερους τύπους:

```
class OneTypeParam<T> {

    private T t;

    public OneTypeParam(T t) {
        this.t = t;
    }

    public void setT(T t) {
        this.t = t;
    }

    public T getT() {
        return t;
    }
}
```

Κώδικας 17.2 *Ορισμός γενίκευσης με μία παράμετρο τύπου*

Στον κώδικα 17.2 ορίζουμε μία γενικευμένη κλάση. Η T αποτελεί μεταβλητή τύπου. Κατά τον ορισμό της κλάσης, η T δεν αντιστοιχεί σε σαφή τύπο. Κατά τη δημιουργία όμως των αντικειμένων είμαστε υποχρεωμένοι να ορίσουμε σαφώς έναν τύπο όπως δείχνει ο παρακάτω κώδικας:

```
OneTypeParam<String> oTP = new OneTypeParam<>("Jim");
```

Το αντικείμενο oTP δημιουργείται αφού ο μεταγλωττιστής παραγάγει μία μη γενικευμένη κλάση στην οποία αντικαθιστά κάθε T με το String. Η διαδικασία αυτή αναφέρεται ως αντικατάσταση τύπου (type erasure). Αντίστοιχα, μία δήλωση του τύπου

```
OneTypeParam<Integer> oTP = new OneTypeParam<>(150);
```

θα ωθήσει τον μεταγλωττιστή να παράξει μία κλάση στην οποία κάθε T αντικαθίσταται από Integer.

Γενικά, για να δημιουργήσουμε ένα αντικείμενο τύπου OneTypeParameter<Integer>, θα πρέπει να καλέσουμε new OneTypeParameter<Integer>. Ωστόσο, όταν ο τύπος μπορεί να υπολογιστεί από τα συμφραζόμενα (context) όπως στα παραδείγματα που ήδη δώσαμε, τότε μπορούμε να χρησιμοποιήσουμε ένα άδειο σύνολο από παραμέτρους τύπων. Το άδειο σύνολο εκφράζεται από τον τελεστή διαμαντιού (diamond

operator) που συμβολίζεται ως <>. Η διαδικασία αυτόματης αναγνώρισης των παραμέτρων τύπων από τα συμφραζόμενα ονομάζεται εξόρυξη τύπου (type inference).
Ένας γενικευμένος τύπος μπορεί να παραμετροποιείται για περισσότερες από μία μεταβλητές τύπου όπως δείχνει ο κώδικας 17.3:

```
class TwoTypeParam<K, V> {

    private K key;
    private V value;

    public TwoTypeParam(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() {
        return key;
    }

    public V getValue() {
        return value;
    }

    public void setKey(K key) {
        this.key = key;
    }

    public void setValue(V value) {
        this.value = value;
    }
}
```

Κώδικας 17.3 *Ορισμός γενίκευσης με δύο παραμέτρους τύπου.*

Έχοντας τον κώδικα 17.3 στη διάθεσή μας, μπορούμε να ορίσουμε αντικείμενα ως εξής:

```
TwoTypeParam<String,Integer>tTP=new
TwoTypeParam<>("Jim",120);
```

Επιπλέον, μια παράμετρος τύπου μπορεί να είναι και η ίδια μια γενίκευση όπως δείχνει η ακόλουθη δήλωση:

```
TwoTypeParam<Integer,OneTypeParam<String>>                t=new
TwoTypeParam<>(5,new OneTypeParam<>("Jim"));
```

Ας σημειωθεί πως στη θέση των μεταβλητών τύπου είναι αποδεκτές μόνο κλάσεις και όχι θεμελιώδεις τύποι.

17.2 Συναρτήσεις παραμετροποιημένες ως προς τύπο

Πέραν των κλάσεων είναι δυνατόν να παραμετροποιηθούν ως προς έναν ή περισσότερους τύπους και οι συναρτήσεις:

```
public class GenericFunctions {

    static <K, V> boolean equalId(TwoTypeParam<K, V> p1, TwoTypeParam<K, V> p2) {
        return p1.getKey().equals(p2.getKey())
                && p1.getValue().equals(p2.getValue());
    }

    public static void main(String[] args) {
```

```
        TwoTypeParam<String, Integer> t1 = new TwoTypeParam<>("Jim", 100),
                t2 = new TwoTypeParam<>("Jim", 100),
                t3 = new TwoTypeParam<>("John", 200);
        System.out.println(equalId(t1, t2));
        System.out.println(equalId(t1, t3));
        TwoTypeParam<String, Double> t4=new TwoTypeParam<>("Jim", 100d);
    }
}
```

Κώδικας 17.4 *Η συνάρτηση equalId παραμετροποιημένη ως προς δύο τύπους*

Όπως φαίνεται στον κώδικα 17.4, το συντακτικό ορισμού συνάρτησης παραμετροποιημένης ως προς τύπο απαιτεί πριν τον τύπο επιστροφής της συνάρτησης να δηλωθεί το σύνολο των μεταβλητών τύπου. Στη συνέχεια οι μεταβλητές τύπου μπορούν να αξιοποιηθούν μέσα στο σώμα της συνάρτησης. Κατά την κλήση αντικαθίστανται όπως και στην περίπτωση των κλάσεων με μεταβλητές των πραγματικών τύπων. Προσέξτε όμως, η κλήση της equalId, όπως φαίνεται στον κώδικα που ακολουθεί, παράγει λάθος μεταγλώττισης.

```
TwoTypeParam<String, Double> t4=new TwoTypeParam<>("Jim",
100d);
System.out.println(equalId(t1,t4));
```

Πράγματι, η equalId διαθέτει δύο μεταβλητές παραμέτρου, την K και την V. Παράλληλα στη λίστα των παραμέτρων της οι δύο TwoTypeParam είναι και οι δύο <K,V>. Οι πραγματικές παράμετροι όμως που περνάμε είναι <String, Integer> και <String, Double>. Σε κάθε κλήση όμως της equal τόσο το K όσο και το V πρέπει να αναφέρονται σε συγκεκριμένο τύπο. Φυσικά, δεν θα υπήρχε πρόβλημα αν περνούσαμε δύο TwoTypeParam<String, Double>.

17.3 Παραμετροποιημένοι τύποι με όρια

Ας υποθέσουμε πως χρειαζόμαστε μία συνάρτηση που λαμβάνει ως παράμετρο μια λίστα από αντικείμενα Person ή μία λίστα από αντικείμενα Student και για κάθε ένα εμφανίζει το όνομά του, το επώνυμό του και την ηλικία του.
Ίσως, η πρώτη σκέψη μας να αποδίδεται από τον κώδικα 17.5.

```
    static void error(List<Person> table) {

        for (Person p : table) {
            System.out.println(p.getfName()+" "+p.getsName()+" "+p.getAge());
        }
    }
```

Κώδικας 17.5 *Εμφάνιση στοιχείων λίστας*

Πράγματι, ο κώδικας τρέχει σωστά με οποιαδήποτε λίστα τύπου List<Person>. Ωστόσο, ο κώδικας δεν μεταγλωττίζεται αν προσπαθήσουμε να περάσουμε ως πραγματική παράμετρο μία List<Student>. Παρότι η κλάση Student είναι παράγωγη της Person, η List<Student> δεν σχετίζεται ιεραρχικά με την List<Person>.

Τη λύση σε αυτό το πρόβλημα προσφέρουν οι παραμετροποιημένοι τύποι με όρια (bounded type parameters). Εκείνο που πραγματικά επιθυμούμε σε σχέση με μία τέτοια συνάρτηση είναι να δέχεται ως παράμετρο μια λίστα από αντικείμενα Person ή μία λίστα από αντικείμενα Student ή γενικότερα μία λίστα από αντικείμενα κλάσης παράγωγης της Person. Αυτό επιτυγχάνεται εύκολα, όπως δείχνει ο ακόλουθος κώδικας:

```
    static <T extends Person> void report(List<T> table) {

        for (Person p : table) {
            System.out.println(p.getfName()+" "+p.getsName()+" "+p.getAge());
        }
    }
```

Κώδικας 17.6 *Εμφάνιση στοιχείων γενικευμένης λίστας*

Η report τώρα δέχεται ως παράμετρο και List<Person> και List<Student>. Ας σημειωθεί πως στην List<Person> μπορεί να υπάρχουν αντικείμενα τύπου Student. Αντίθετα στην List<Student> δεν μπορεί να υπάρχουν αντικείμενα τύπου Person.

Επιπλέον, η λέξη extends εδώ έχει διαφοροποιημένη έννοια, καθώς μπορεί να ακολουθείται από κλάση ή διεπαφή. Στην περίπτωση που ακολουθείται από διεπαφή σημαίνει πως απαιτείται κλάση που υλοποιεί τη διεπαφή.

17.4 Η κληρονομικότητα στις γενικεύσεις

Η κληρονομικότητα στις γενικεύσεις συχνά αποτελεί ζήτημα που προκαλεί σύγχυση. Ας το δούμε με ένα παράδειγμα. Έστω ότι έχουμε μια συνάρτηση f:

```
static void f(Number k) {}
```

Οι κλήσεις

```
f(8);
f(8d)
```

είναι και οι δύο αποδεκτές. Τόσο η Integer όσο και η Double είναι παράγωγες της Number και όπως έχουμε μάθει αντικείμενο παράγωγης είναι αποδεκτό εκεί όπου αναμένεται αντικείμενο γονικής.
Παρόμοια, η συνάρτηση g

```
static <T extends Number> void g(T t) {}
```

μπορεί να κληθεί ως ακολούθως

```
g(8);
g(8d);
```

Η συνάρτηση

```
static void k(List<Number> l) {}
```

μπορεί να κληθεί ως

```
k(new ArrayList<Number>());
k(new LinkedList<Number>());
```

Όμως η κλήση

```
k(new ArrayList<Integer>());
```

δεν είναι σωστή.
Πράγματι η ArrayList<Number> είναι παράγωγη της List<Number>. Το ίδιο ισχύει και για την LinkedList<Number>. Ωστόσο η ArrayList<Integer> δεν είναι παράγωγη της List<Number>, παρότι η ArrayList είναι παράγωγη της List και η Integer παράγωγη της Number. Ίσως εκ πρώτης όψεως να είναι λίγο δύσκολο να το κατανοήσει κανείς, γίνεται όμως ευκολότερο αν ανακαλέσει πώς λειτουργεί ο μεταγλωττιστής με τις γενικεύσεις. Έχουμε πει πως όταν παράγει ένα αντικείμενο κάποιας γενίκευσης, αντικαθιστά τις μεταβλητές τύπου με την εξειδίκευση κατά τη δημιουργία των αντικειμένων. Για παράδειγμα, η ArrayList<Number> σε γενικές γραμμές έχει την ακόλουθη μορφή:

```
class ArrayListNumber {
     Number[] store;
     …
```

```
        …
    }
```

Αντίθετα, η ArrayList<Integer> έχει τη μορφή

```
    class ArrayListInteger {
        Integer[] store;
        …
        …
    }
```

Είναι προφανές πως οι δύο κλάσεις δεν συνδέονται με σχέσεις ιεραρχίας. Γενικότερα ισχύει πως αν έχουμε δύο κλάσεις A και B, ο τύπος MyClass<A> δεν σχετίζεται ιεραρχικά με τον τύπο MyClass<B>, ακόμη και αν η κλάση B είναι παράγωγη της A. Ο κοινός πρόγονος των MyClass<A> και MyClass<B> είναι η κλάση Object [1].

Γενικότερα, όταν μια γενίκευση κληρονομεί μία άλλη ή υλοποιεί μια γενικευμένη διεπαφή, η μεταξύ τους σχέση κληρονομικότητας διατηρείται μόνο για την ίδια παράμετρο τύπου. Για παράδειγμα, η ArrayList<E> υλοποιεί την List<E>. Επομένως, ένα ArrayList<Integer> είναι ένα List<Integer>, δεν είναι όμως ένα List<Number>, εξάλλου δεν είναι καν ArrayList<Number>.

17.5 Ο χαρακτήρας μπαλαντέρ

Ας υποθέσουμε τώρα πως χρειαζόμαστε μια συνάρτηση που να υπολογίζει το άθροισμα μιας λίστας αριθμών. Η συνάρτηση θα πρέπει να δουλεύει με όλες τις κλάσεις που υλοποιούν τη διεπαφή List αλλά και με όλους τους τύπους αριθμών, δηλαδή με όλες τις παράγωγες κλάσεις της Number. Με άλλα λόγια, θα μας εξυπηρετούσε μία συνάρτηση που μπορεί να λάβει ως παράμετρο και ArrayList<Double> και LinkedList<Integer> αλλά και όλους τους υπόλοιπους συνδυασμούς λιστών και αριθμών.

Τη δυνατότητα αυτή μας παρέχει ο χαρακτήρας μπαλαντέρ (WildCard) που αναπαρίσταται με το αγγλικό ερωτηματικό:

```
static double sum(List<? extends Number> l) {
    double sum = 0d;
    for (int i = 0; i < l.size(); i++) {
        sum += l.get(i).doubleValue();
    }
    return sum;
}
```

Κώδικας 17.7 *Αξιοποίηση του χαρακτήρα μπαλαντέρ με άνω όριο*

Η έκφραση List<? extends Number> στη λίστα παραμέτρων της sum σημαίνει πως η συνάρτηση δέχεται ως παραμέτρους αντικείμενο οποιασδήποτε κλάσης υλοποιεί την List, ενώ τα στοιχεία του είναι Number ή παράγωγης της Number. Έτσι, οι κλήσεις στον κώδικα 17.8 είναι όλες σωστές:

```
public static void main(String[] args) {
    List<Double> ld=new ArrayList<>();
    List<Integer> li=new ArrayList<>();
    List<Number> ln=new LinkedList<>();
    for (int i=0; i<10; i++) {
        ld.add((double)i);
        li.add(i);
        ln.add(i);
    }
    System.out.println(sum1(ld));
    System.out.println(sum1(li));
    System.out.println(sum1(ln));
}
```

Κώδικας 17.8 *Κλήσεις της γενικευμένης συνάρτησης sum*

Ας σημειωθεί πως εναλλακτική προσέγγιση σε αυτό το πρόβλημα αποτελεί μία συνάρτηση με την ακόλουθη διεπαφή:

```
static <T extends Number> double sum1(List<T> l)
```

Στο παράδειγμα που αναπτύχθηκε στους κώδικες 17.7 και 17.8 περιορίσαμε τα στοιχεία της λίστας ώστε τα στοιχεία της να είναι Number ή παράγωγες κλάσεις. Με άλλα λόγια, δώσαμε το άνω όριο (upper bound) των κλάσεων που είναι αποδεκτές ως τύπος της λίστας. Αυτό ήταν αναγκαίο, καθώς μέσα στο σώμα της sum εκτελούμε προσθέσεις με αυτά τα στοιχεία. Αν όμως θέλαμε μια επεξεργασία που δεν απαιτεί συγκεκριμένο τύπο θα μπορούσαμε να χρησιμοποιήσουμε τον χαρακτήρα μπαλαντέρ χωρίς όρια όπως δείχνει ο κώδικας 17.9:

```
    static void prt(List<?> l) {
        for (int i=0; i<l.size(); i++) {
            System.out.println(l.get(i));
        }
    }
```

Κώδικας 17.9 *Αξιοποίηση του χαρακτήρα μπαλαντέρ χωρίς όρια*

Ας υποθέσουμε τώρα πως χρειαζόμαστε μία συνάρτηση που να εισάγει αντικείμενα τύπου Integer σε μία λίστα. Εκ πρώτης όψεως φαίνεται εύκολη απαίτηση, εκτός αν λάβουμε υπόψη ότι σε αντικείμενα Integer μπορούμε να αναφερόμαστε με μεταβλητές τύπου Number αλλά και με μεταβλητές τύπου Object, δηλαδή με μεταβλητές που ο τύπος τους είναι Integer ή κάποιος πρόγονος της Integer. Επομένως, η συνάρτησή μας θα πρέπει να ανταποκρίνεται σωστά με List<Integer>, με List<Number> και με List<Object>.
Μια τέτοια απαίτηση μπορούμε να υλοποιήσουμε με χρήση μπαλαντέρ με κάτω όριο (lower bound).

```
import java.util.ArrayList;
import java.util.List;

public class WCLowBounds {

    static void insert(List<? super Integer> l) {
        Integer i = 3;
        l.add(i);
    }

    public static void main(String[] args) {
        ArrayList<Integer> li = new ArrayList<>();
        ArrayList<Number> ln = new ArrayList<>();
        ArrayList<Object> lo = new ArrayList<>();
        insert(li);
        insert(ln);
        insert(lo);
    }
}
```

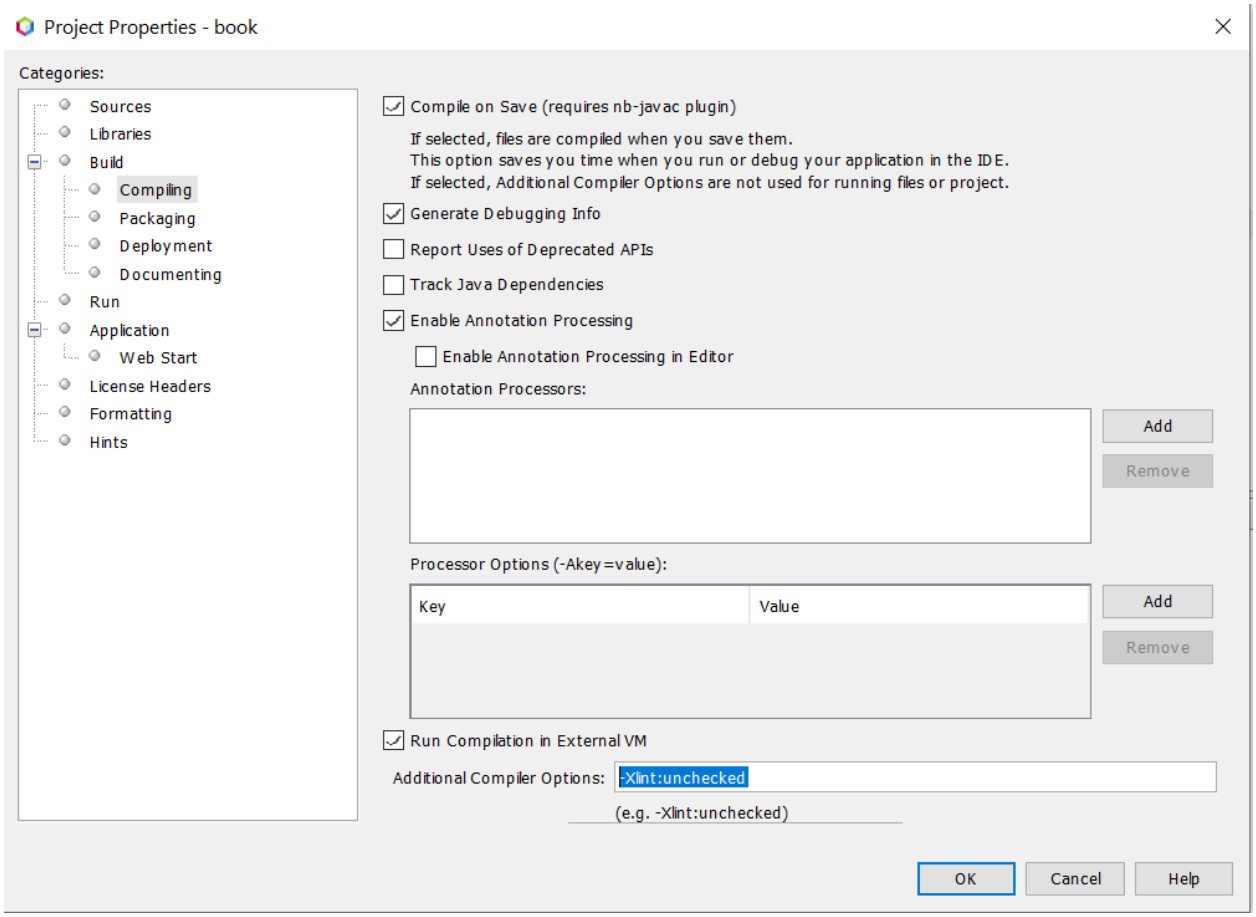
Κώδικας 17.10 *Αξιοποίηση του χαρακτήρα μπαλαντέρ με κάτω όριο*

Στον κώδικα 17.10 η συνάρτηση insert δηλώνει την παράμετρό της με χρήση μπαλαντέρ με κάτω όριο. Όπως φαίνεται στον κώδικα, η δήλωση γίνεται με τη δεσμευμένη λέξη super. Η insert μπορεί να λάβει ως παράμετρο οποιαδήποτε λίστα από Integers, Numbers ή Objects. Στη λίστα όμως επιτρέπεται μόνο η εισαγωγή Integer.

Ανακεφαλαιώνοντας, επισημαίνουμε τα εξής: Η List<Number> δεν είναι γονική της List<Integer>. Και οι δύο κλάσεις είναι παράγωγες της List<?>. Όμως η List<? extends Number> είναι γονική της List<? extends Integer>.

17.6 Μη Ασφαλείς Λειτουργίες

Σε μερικές περιπτώσεις έχουμε χρησιμοποιήσει μη ασφαλείς λειτουργίες (unsafe operations). Ο μεταγλωττιστής της Java επιτρέπει τις μη ασφαλείς λειτουργίες, καθώς σε πολλές περιπτώσεις είναι αναγκαίες. Εξ ορισμού δεν παράγει καν προειδοποιήσεις. Αν θέλετε να δείτε τις μη ασφαλείς λειτουργίες του κώδικά σας θα πρέπει να μεταγλωττίσετε με την παράμετρο -Xlint:unchecked. Μεταβείτε στα Properties του project, επιλέξτε Compiling και ενημερώστε τις επιπρόσθετες επιλογές μεταγλώττισης (Additional Compiler Options) όπως δείχνει η εικόνα 17.1:



Εικόνα 17.1 *Επιπρόσθετες επιλογές μεταγλώττισης*

Τι είναι όμως ακριβώς οι μη ασφαλείς λειτουργίες; Θα το εξηγήσουμε με ένα παράδειγμα. Έστω ότι θέλουμε να υλοποιήσουμε μία γενικευμένη συνάρτηση ταξινόμησης πινάκων χωρίς να χρησιμοποιήσουμε την sort της Arrays. Προκειμένου να ταξινομηθούν τα στοιχεία ενός πίνακα θα πρέπει να είναι συγκρίσιμα μεταξύ τους. Οι κλάσεις που υλοποιούν τη διεπαφή Comparable<T> παρέχουν τη δυνατότητα σύγκρισης των αντικειμένων τους. Πολλές προκαθορισμένες κλάσεις της Java υλοποιούν την Comparable<T>. Αυτή καθορίζει τη φυσική σειρά (natural order) των αντικειμένων. Μπορείτε να βρείτε περισσότερες πληροφορίες στην τεκμηρίωση της Java. Πάντως, η συνάρτησή μας θα πρέπει να λαμβάνει πίνακες από αντικείμενα Comparable, όπως δείχνει ο κώδικας 17.11:

```
    private static <E> void swap(E[] a, int i, int j) {
        if (i != j) {
            E temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }

    public static <E extends Comparable<E>> void sort(E[] a) {
        for (int i = 0; i < a.length - 1; i++) {
            int smallest = i;
```

```
            for (int j = i + 1; j < a.length; j++) {
                if (a[j].compareTo(a[smallest]) <= 0) {
                    smallest = j;
                }
            }
            swap(a, i, smallest);
        }
    }
```

Κώδικας 17.11 *Γενικευμένη έκδοση της sort πινάκων*

Ας υποθέσουμε πως έχουμε στη διάθεσή μας και την κλάση Tmp του κώδικα 17.12.

```
class Tmp implements Comparable<Tmp> {

    private final String id;
    private final String description;

    public Tmp(String id, String description) {
        this.id = id;
        this.description = description;
    }

    @Override
    public int compareTo(Tmp o) {
        return id.compareTo(o.id);
    }

    @Override
    public String toString() {
        return id;
    }
}
```

Κώδικας 17.12 *Η Tmp που υλοποιεί την διεπαφή Comparable.*

Σε αυτήν την περίπτωση μπορούμε να δημιουργήσουμε πίνακες ως εξής:

```
Integer[] ints = new Integer[10];
Double[] doubles = new Double[10];
Tmp[] tmps = new Tmp[10];
```

Αν δώσουμε τιμές στα στοιχεία σε κάθε έναν από τους πίνακες ints, doubles και tmps και στη συνέχεια καλέσουμε την sort για κάθε πίνακα, θα διαπιστώσουμε ότι όλοι οι πίνακες ταξινομούνται σωστά χωρίς κανένα πρόβλημα. Θεωρήστε τώρα τον πίνακα

```
Comparable[] l=new Comparable[3];
```

που περιλαμβάνει στοιχεία ως εξής:

```
l[0]=10;
l[1]=2d;
l[2]=new Tmp(Integer.toString(1), Integer.toString(2));
```

και την κλήση

```
sort(tmps);
```

Ο κώδικας θα πέσει με java.lang.ClassCastException. Η κλήση sort(tmps) συνιστά μη ασφαλή κλήση, διότι αν προσέξετε τη διεπαφή της sort θα δείτε πως απαιτεί πίνακα στοιχείων τύπου E ο οποίος να είναι Comparable.

Ωστόσο είναι συγκεκριμένος τύπος. Αντίθετα στον πίνακα l περιλαμβάνονται μεν Comparables διαφορετικών ωστόσο τύπων. Έτσι, όταν μέσα στην sort η compareTo επιχειρεί να συγκρίνει ένα αντικείμενο τύπου Integer με ένα τύπου Double ή τύπου Tmp, προσπαθεί να μετατρέψει τον τύπο Double ή Tmp σε Integer, αποτυγχάνει και πέφτει με ClassCastException. Αν ωστόσο όλα τα στοιχεία του πίνακα l αφορούν την ίδια κλάση δεν υπάρχει πρόβλημα παρότι η κλήση συνεχίζει να χαρακτηρίζεται ως μη ασφαλής. Εναπόκειται επομένως στην ευθύνη του κώδικα-πελάτη να καλέσει την sort με σωστό τρόπο.
Σε γενικές γραμμές θα πρέπει να αποφεύγονται οι μη ασφαλείς λειτουργίες. Σε πολλές περιπτώσεις όμως είναι αναγκαίες. Για παράδειγμα, στον κώδικα 15.8, η συνάρτηση loadCars περιέχει τη γραμμή

```
ArrayList<Car> loaded = (ArrayList<Car>) in.readObject();
```

Η readObject επιστρέφει αντικείμενο τύπου Object το οποίο μετατρέπουμε σε ArrayList<Car>. Ο μεταγλωττιστής δεν μπορεί να γνωρίζει αν η μετατροπή αυτή είναι σωστή. Με αυτήν την έννοια, η μετατροπή χαρακτηρίζεται ως μη ασφαλής. Ωστόσο, είναι αναγκαία εφόσον είναι ο μόνος τρόπος για να φορτώσουμε στη μνήμη ένα σειριοποιημένο ArrayList<Car>. Πάλι εναπόκειται στην ευθύνη του κώδικα πελάτη να κάνει τη σωστή μετατροπή, διαφορετικά θα λάβει ClassCastExeption.
Ένα άλλο παράδειγμα υπάρχει στον κώδικα 12.20. Προσέξτε πως στην αρχή της κλάσης MyHashMap υπάρχει η γραμμή

```
private final ArrayList<Person>[] store = new ArrayList[SIZE];
```

Η Java δεν επιτρέπει τη δημιουργία γενικευμένων πινάκων. Ο τύπος του πίνακα store είναι ArrayList<Person>, ωστόσο, ο πραγματικός πίνακας που δημιουργείται είναι ArrayList (raw type). Προσέξτε ότι κατά τη δημιουργία δεν χρησιμοποιήθηκε ο τελεστής διαμαντιού, διαφορετικά η γραμμή θα είχε ως εξής:

```
private final ArrayList<Person>[] store = new
ArrayList<>[SIZE];
```

Ωστόσο, η τελευταία αυτή γραμμή δεν περνάει μεταγλώττιση. Έτσι, όταν χρειαζόμαστε έναν πίνακα στον οποίο θα αποθηκεύουμε γενικεύσεις, στην πράξη δημιουργούμε έναν πίνακα από Object. Καθώς η Object είναι γονική όλων των κλάσεων, είναι και γονική κάθε γενίκευσης. Με άλλα λόγια σε μία αναφορά τύπου Object μπορούμε να εκχωρήσουμε οποιαδήποτε γενίκευση. Επομένως, οι πίνακες στους οποίους αποθηκεύουμε γενικεύσεις μάς εξαναγκάζουν να χρησιμοποιούμε μη ασφαλείς λειτουργίες. Ας σημειωθεί πως η ίδια η προκαθορισμένη κλάση Collections δεν μεταγλωττίζεται χωρίς προειδοποιήσεις (warnings) για μη ασφαλείς λειτουργίες.

17.7 Λυμένες Ασκήσεις

17.7.1 MyHashMap<K,V>

Να υλοποιηθεί η γενίκευση MyHashMap<K,V> χωρίς να χρησιμοποιηθεί η προκαθορισμένη γενίκευση HashMap<K,V>. Ο εσωτερικός αποθηκευτικός χώρος της MyHashMap θα πρέπει να είναι ένας πίνακας σε κάθε θέση του οποίου θα τοποθετείται ένα ArrayList στο οποίο θα εισάγονται οι συσχετίσεις K, V ανάλογα με τον κωδικό κατακερματισμού τους. Ο εσωτερικός πίνακας θα δημιουργείται με αρχικό μέγεθος 16. Επιπλέον, θα οριστεί ένας συντελεστής φόρτωσης. Όταν οι κατειλημμένες θέσεις του εσωτερικού πίνακα υπερβαίνουν το μέγεθος του πίνακα επί τον συντελεστή φόρτωσης, το μέγεθος του εσωτερικού πίνακα θα πρέπει να διπλασιάζεται και οι συσχετίσεις να επανατοποθετούνται, δηλαδή να γίνεται επανακατακερματισμός (rehashing). Σε ένα MyHashMap δεν επιτρέπεται η εισαγωγή συσχετίσεων με το ίδιο κλειδί. Η MyHashMap θα πρέπει να υποστηρίζει τις ακόλουθες μεθόδους:

```
public void clear()
```

Διαγράφει όλες τις συσχετίσεις από αυτό το αντικείμενο

```
public V put(K key, V value)
```